

Operating Systems – Final

June 18, 2013

Total: 100 points

Handwritten notes:
 User
 ↓
 Scheduler
 ↓
 Upcall
 ↓
 Critical code
 ↓
 CPU

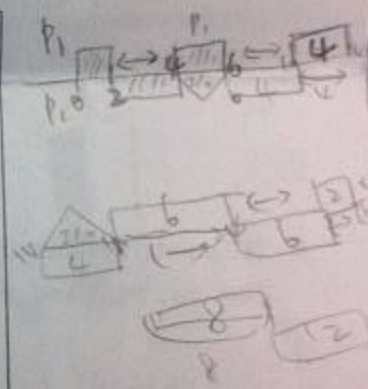
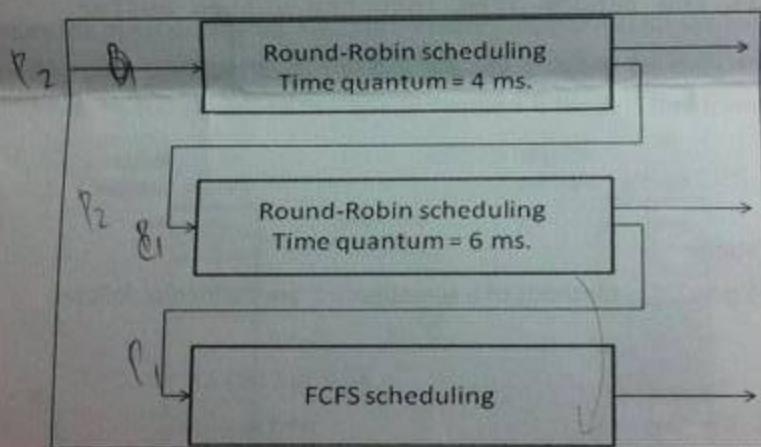
1. Scheduler activations:

- (a) (5 points) Explain the problem of many-to-many thread model without using scheduler activations.
- (b) (10 points) Show how scheduler activations work when a user thread is making a blocking system call.
 You **HAVE TO** mention **upcall handler**, **scheduler**, and their roles in a thread library.

2. Scheduling – Consider the following two processes P_1 and P_2 , each of which consists of two CPU bursts and one I/O burst. Assume that P_1 and P_2 arrive at time 0 and 2 (millisecond) respectively:

P_1 :	CPU burst: 8 milliseconds.	I/O burst: 4 milliseconds.	CPU burst: 8 milliseconds.
P_2 :	CPU burst: 2 milliseconds.	I/O burst: 2 milliseconds.	CPU burst: 16 milliseconds.

- (a) (5 points) Draw a Gantt chart illustrating the execution of the processes using the following **preemptive multilevel feedback-queue scheduling algorithm**. Note that a high-level process will preempt a low-level process and a process will be scheduled to a lower level if it is preempted.



- (b) (5 points) What is the average waiting time for the scheduling algorithm?

3. (15 points) Producer-Consumer Problem – Complete the following producer thread using **Pthreads mutex locks and semaphores**

```
// global (share) variables
int buffer[5];           // shared buffer
pthread_mutex_t mutex;   // mutex for shared data access
sem_t empty;             // the number of empty buffers
sem_t full;              // the number of full buffers
int in, out;             // in: the next free position in the buffer
                        // out: the first full position in the buffer
```

```

int main(void) {
    //Initialize mutex and counting semaphoresv1 empty and full
    mutex
    for {
        /* Initialize buffer */
        /* Create producer thread(s) */
        /* Create consumer thread(s) */
        /* Sleep */
        /* Exit */
    }
}

```

```

void *producer(void *param) {
    int random_item;    // the produced item
    int sleep_interval; // the time of thread sleeping

    while(TRUE) {
        sleep_interval = rand() % 5;
        sleep(sleep_interval); // sleep for a rand period of time

        random_item = rand(); // generate a random integer

        // insert the random_item into the shared buffer
    }
}

```

4. (10 points) Synchronization:

The wait() and signal() methods of a semaphore S are defined as follows:

```

wait(S) {
    while (S <= 0)
        ; // no-operation
    S--;
}

signal(S) {
    S++;
}

```

In wait(), the testing of S (i.e., $S \leq 0$) and its possible modifications ($S--$) must be atomic (executed without interruption). Use an execution sequence to show that if $S--$ is not atomic, then the mutual exclusion of critical section cannot be preserved. The initial value of S is 1.

Hint: $S--$ may be implemented in machine language as

```

register1 = S
register1 = register1 - 1
S = register1

```

Deadlocks:

- (a) (10 points) Consider the following snapshot of a SAFE system; show why granting request (3,2,1) from p_0 will make the system unsafe using the banker's algorithm.

	Allocation			Max			Available		
	A	B	C	A	B	C	A	B	C
P_0	0	1	0	8	5	3	3	3	2
P_1	2	0	0	3	2	2			
P_2	3	0	2	9	0	2			
P_3	2	1	1	2	2	2			
P_4	0	0	2	4	3	3			

Need

0 1 1
2 1 1
4 2 2
1 2 1

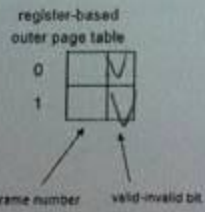
0 1 1
2 1 1
4 2 2
1 2 1

- (b) (5 points) Explain why a deadlock **MAY NOT** occur even if we grant the above request. And under which condition, a deadlock occurs.

6. Paging – Consider a paging system with a 4-bit logical address space, 4 bytes per page, and 32 bytes physical memory.

- (a) (5 points) If each page table entry is 2 bytes long (the first byte for frame number & the second byte for protection bits), then how many bytes are required for a page table?
- (b) (10 points) Suppose the system employs the technique of **TWO-LEVEL PAGING** and registers are used to store the outer page table. Allocate memory (i.e., frames) for the following process and the corresponding page table. Note: (i) You first allocate memory for the page table, and then the process. (ii) Free frames are stored in a queue. Frame 5 is the first free frame.

logical address	content
0	A
1	B
2	C
3	D
4	E
5	F
6	G
7	H
8	I
9	J
10	K
11	L
12	M
13	N
14	O
15	P



- free-frame list
- 5
 - 0
 - 2
 - 7
 - 3
 - 4

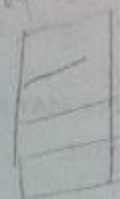
physical address	content
0	I
1	F
2	G
3	H
4	
5	
6	
7	
8	I
9	J
10	K
11	L
12	
13	
14	
15	
16	
17	
18	
19	
20	A
21	B
22	C
23	D
24	
25	
26	
27	
28	M
29	N
30	O
31	P



7. Virtual Memory

- (a) (10 points) Describe the actions taken by the operating system when a user process tries to access an invalid memory page. You need to consider whether the reference is invalid or it is a page fault. You also need to consider page replacement when there is no free frame for a page fault. In addition, a modify bit is provided to improve page-fault service time.
- (b) Consider the following logical memory references to a system with 2KB physical memory which is partitioned into 4 frames of the same size. How many page faults would occur for (5 points) the optimal page-replace algorithm and (5 points) the LRU page replacement algorithm?

001010001001
 010011100011
 011101110100
 100000000111
 010111101000
 001000111111
 101101011111
 110111111001
 010010000000
 001101111000
 010111111111
 011101111011
 111000110000
 110100011111
 011010100000
 010000010111
 001000000001
 010000010011
 011111101101
 110010011001



21 1111 1111
 21 1111 1111